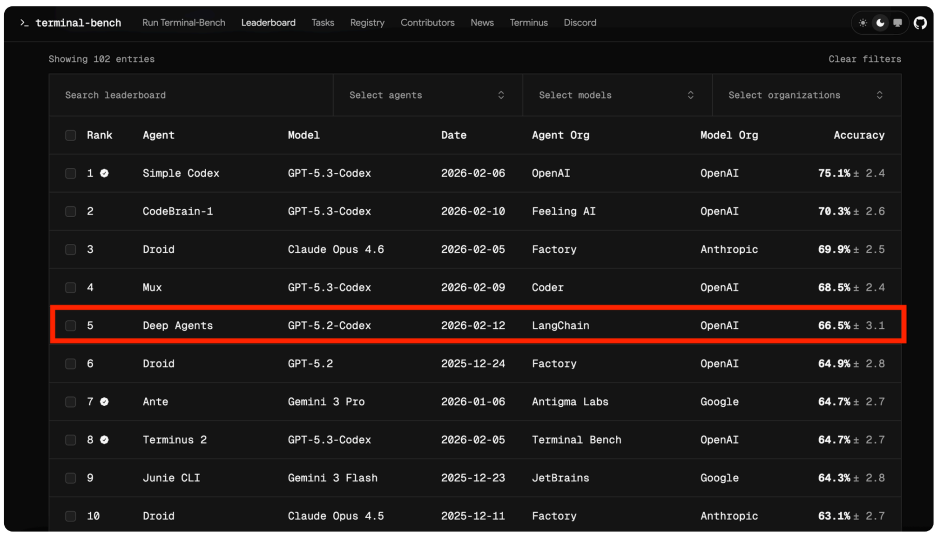
Deep Agents | Наблюдаемость и оценка | Архитектура агента

Улучшение работы Deep Agents с помощью harness engineering


Вивек Триведи
17 февраля 2026

8 мин

← Вернуться к блогу



Rank	Agent	Model	Date	Agent Org	Model Org	Accuracy
1	Simple Codex	GPT-5.3-Codex	2026-02-06	OpenAI	OpenAI	75.1% ± 2.4
2	CodeBrain-1	GPT-5.3-Codex	2026-02-10	Feeling AI	OpenAI	70.3% ± 2.6
3	Droid	Claude Opus 4.6	2026-02-05	Factory	Anthropic	69.9% ± 2.5
4	Mux	GPT-5.3-Codex	2026-02-09	Coder	OpenAI	68.5% ± 2.4
5	Deep Agents	GPT-5.2-Codex	2026-02-12	LangChain	OpenAI	66.5% ± 3.1
6	Droid	GPT-5.2	2025-12-24	Factory	OpenAI	64.9% ± 2.8
7	Ante	Gemini 3 Pro	2026-01-06	Antigma Labs	Google	64.7% ± 2.7
8	Terminus 2	GPT-5.3-Codex	2026-02-05	Terminal Bench	OpenAI	64.7% ± 2.7
9	Junie CLI	Gemini 3 Flash	2025-12-23	JetBrains	Google	64.3% ± 2.8
10	Droid	Claude Opus 4.5	2025-12-11	Factory	Anthropic	63.1% ± 2.7

TLDR: наш агент-программист поднялся с 30-го на 5-е место в Terminal Bench 2.0. Мы изменили только «обвязку». Вот наш подход к «обвязке» (тизер: самопроверка и трассировка очень помогают).

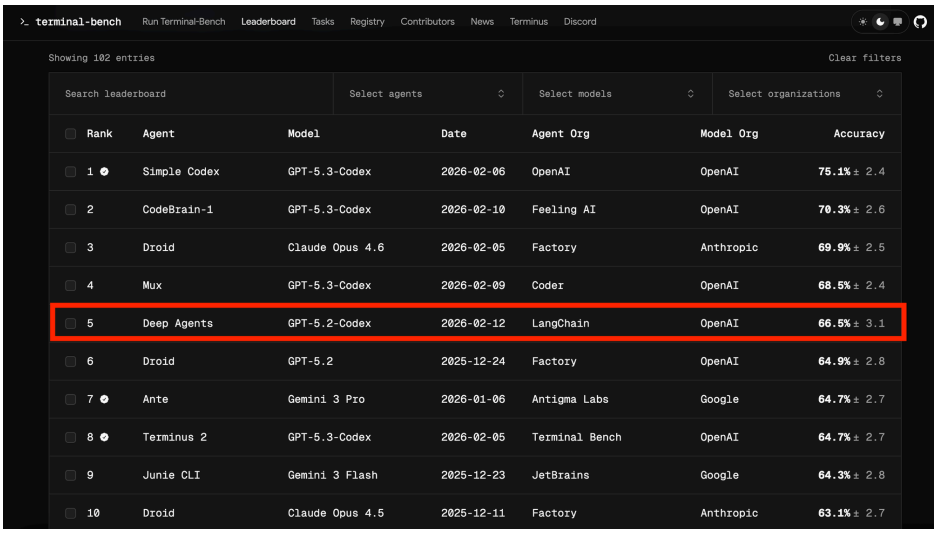
Цель «обвязки»

Цель «обвязки» — адаптировать изначально «колючий» интеллект модели для решения важных для нас задач. **«Обвязка»** — это про системы, в которых вы создаете инструменты вокруг модели для оптимизации таких целей, как производительность задач, эффективность токенов, задержка и т. д. К проектным решениям относятся системная подсказка, выбор инструмента и последовательность выполнения.

Но как изменить систему, чтобы улучшить своего агента?

В LangChain мы используем Traces для понимания причин сбоев в работе агентов в больших масштабах. Современные модели — это в значительной степени «черные ящики», их внутренние механизмы сложно интерпретировать. Но мы можем видеть их входные и выходные данные в текстовом формате, что мы и используем в наших циклах совершенствования.

Мы использовали простой подход для итеративного улучшения deepagents-cli (нашего агента для написания кода) `13.7 points` с `52.8` до `66.5` на Terminal Bench 2.0. Мы лишь немного подправили фреймворк и оставили модель без изменений, `gpt-5.2-codex` .



Rank	Agent	Model	Date	Agent Org	Model Org	Accuracy
1	Simple Codex	GPT-5.3-Codex	2026-02-06	OpenAI	OpenAI	75.1% ± 2.4
2	CodeBrain-1	GPT-5.3-Codex	2026-02-10	Feeling AI	OpenAI	70.3% ± 2.6
3	Droid	Claude Opus 4.6	2026-02-05	Factory	Anthropic	69.9% ± 2.5
4	Mux	GPT-5.3-Codex	2026-02-09	Coder	OpenAI	68.5% ± 2.4
5	Deep Agents	GPT-5.2-Codex	2026-02-12	LangChain	OpenAI	66.5% ± 3.1
6	Droid	GPT-5.2	2025-12-24	Factory	OpenAI	64.9% ± 2.8
7	Ante	Gemini 3 Pro	2026-01-06	Antigma Labs	Google	64.7% ± 2.7
8	Terminus 2	GPT-5.3-Codex	2026-02-05	Terminal Bench	OpenAI	64.7% ± 2.7
9	Junie CLI	Gemini 3 Flash	2025-12-23	JetBrains	Google	64.3% ± 2.8
10	Droid	Claude Opus 4.5	2025-12-11	Factory	Anthropic	63.1% ± 2.7

Настройка эксперимента и «настройки» фреймворка

Мы использовали Terminal Bench 2.0 — ставший стандартом бенчмарк для оценки агентного программирования. Он включает 89 задач в таких областях, как машинное обучение, отладка и биология. Мы используем Harbor для организации запусков. Он запускает песочницы (Daytona), взаимодействует с нашим циклом агента и выполняет проверку и подсчет баллов.

Каждое действие агента сохраняется в LangSmith. Сюда также входят такие показатели, как задержка, количество токенов и затраты.

Регуляторы, которые мы можем повернуть

В агентской инфраструктуре много элементов: системные подсказки, инструменты, хуки/промежуточное ПО, навыки, делегирование субагентов, системы памяти и многое другое. Мы намеренно сужаем пространство для оптимизации и фокусируемся на трех элементах: **системной подсказке, инструментах** и **промежуточном ПО** (так мы называем хуки, связанные с вызовами моделей и инструментов).

Мы начинаем с подсказки по умолчанию и стандартных инструментов + промежуточного ПО. При использовании GPT-5.2-Codex этот вариант дает 52,8 %. Хороший результат, чуть ниже 30-го места в сегодняшней таблице лидеров, но есть куда расти.

Run	What Changed	Score
Baseline	Baseline prompt for coding, file system tools, planning, etc.	52.8%
+ Custom Prompt & Middleware	Focus on a Build-Verify Loop, Inject environment context. Add loop protection, and timeout warnings	63.6%
+ Adaptive reasoning	xhigh+high reasoning	66.5%

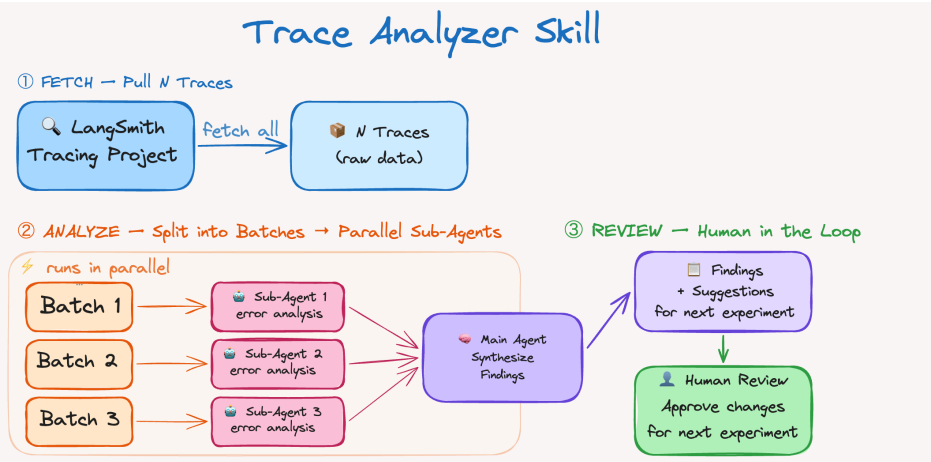
The Trace Analyzer Skill

We wanted trace analysis to be repeatable so we made it into an Agent Skill. This serves as our recipe to **analyze errors across runs and make improvements to the harness**. The flow is:

1. Fetch experiment traces from LangSmith
2. Spawn parallel error analysis agents → main agent synthesizes findings + suggestions
3. Aggregate feedback and make targeted changes to the harness.

This works similarly to boosting which focuses on mistakes from previous runs. A human can be pretty helpful in Step 3 (though not required) to verify and discuss proposed changes. Changes that overfit to a task are bad for generalization and can lead to regressions in other Tasks.

Automated trace analysis saves hours of time and made it easy to quickly try experiments. We'll be publishing this skill soon, we're currently testing it for prompt optimization generally.



What Actually Improved Agent Performance

Automated Trace analysis allowed us to debug where agents were going wrong. Issues included reasoning errors, not following task instructions, missing testing and verification, running out of time, etc. We go into these improvements in more details in the sections below.

Build & Self-Verify

Today's models are exceptional self-improvement machines.

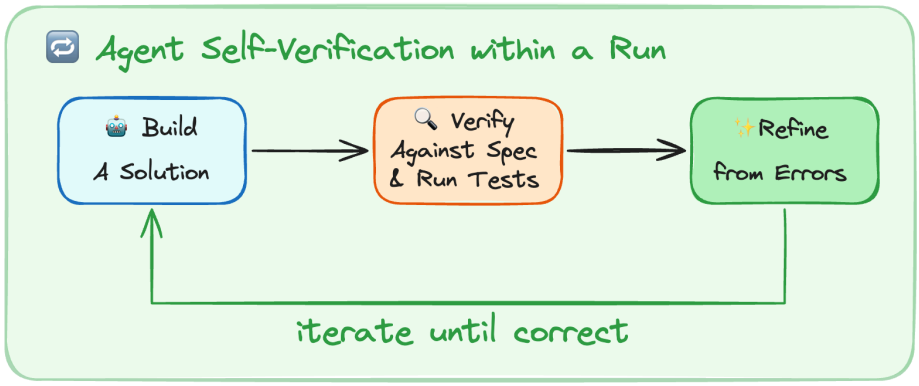
Self-verification allows agents to self-improve via feedback within a run. However, they don't have a natural tendency to enter this **build-verify loop.**

The most common failure pattern was that the agent wrote a solution, re-read its own code, confirmed it looks ok, and stopped. Testing is a key part of autonomous agentic coding. It helps test for overall correctness and simultaneously gives agents signal to hill-climb against.

We added guidance to the system prompt on how to approach problem solving.

1. **Planning & Discovery:** Read the task, scan the codebase, and build an initial plan based on the task specification and how to verify the solution.
2. **Build:** Implement the plan with verification in mind. Build tests, if they don't exist and test both happy paths and edge cases.
3. **Verify:** Run tests, read the full output, compare against what was asked (not against your own code).
4. **Fix:** Analyze any errors, revisit the original spec, and fix issues.

We really focus on testing because it powers the changes in every iteration. We found that alongside prompting, deterministic context injection helps agents verify their work. We use a `PreCompletionChecklistMiddleware` that intercepts the agent before it exits and reminds it to run a verification pass against the Task spec. This is similar to a Ralph Wiggum Loop where a hook forces the agent to continue executing on exit, we use this for verification.



Giving Agents Context about their Environment

Part of harness engineering is **building a good delivery mechanism for context engineering.** Terminal Bench tasks come with directory structures, built-in tooling, and strict timeouts.

1. **Directory Context & Tooling:** A `LocalContextMiddleware` runs on agent start to map the `cwd` and other parent+children directories. We run `bash` commands to find tools like `Python` installations. Context discovery and search are error prone, so injecting context reduces this error surface and helps **onboard the agent into its environment.**
2. **Teaching Agents to Write Testable Code:** Agents don't know how their code needs to be testable. We add prompting say their work will be measured against programatic tests, similar to when committing code. For example, Task specs that mention file paths should be followed exactly so the solutions works in an automated scoring step. Prompting that stresses edge-cases helps the agent avoid only checking "happy path" cases. Forcing models to conform to testing standards is a powerful strategy to avoid "slop buildup" over time.
3. **Time Budgeting:** We inject time budget warnings to nudge the agent to finish work and shift to verification. Agents are famously bad at time estimation so this heuristic helps in this environment. Real world coding usually doesn't have strict time limits, but without adding any knowledge of constraints, agents won't work within time bounds.

The more that agents know about their environment, constraints, and evaluation criteria, the better they can autonomously self-direct their work.

The purpose of the harness engineer: prepare and deliver context so agents can autonomously complete work.

Encouraging Agents to Step Back & Reconsider Plans

Agents can be myopic once they've decided on a plan which results in "doom loops" that make small variations to the same broken approach (10+ times in some traces).

We use a `LoopDetectionMiddleware` that tracks per-file edit counts via tool call hooks. It adds context like "...consider reconsidering your approach" after `N` edits to the same file. This can help agents recover from doom loops, though the model can continue down the same path if it thinks it's correct.

Important note. This is a design heuristic that engineers around today's perceived model issues. As models improve, these guardrails will likely be unnecessary, but today helps agents execute correctly and autonomously.

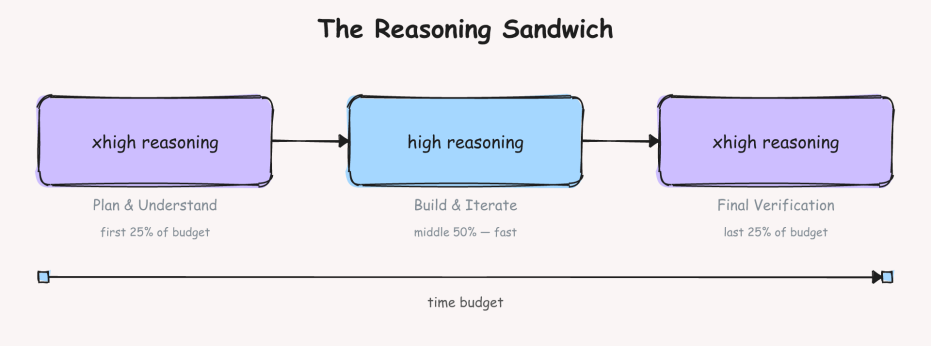
Choosing How Much Compute to Spend on Reasoning

Reasoning models can run autonomously for hours so we have to decide how much compute to spend on every subtask. You can use the max reasoning budget on every task, but most work can benefit from optimizing reasoning compute spend.

Terminal Bench timeout limits create a tradeoff. More reasoning helps agents evaluate each step, but can burn over `2x` more tokens/time. `gpt-5.2-codex` has 4 reasoning modes, `low`, `medium`, `high`, and `xhigh`.

We found that reasoning helps with planning to fully understand the problem, some Terminal Bench tasks are very difficult. A good plan helps get to a working solution more quickly.

Later stage verification also benefits from more reasoning to catch mistakes and get a solution submitted. As a heuristic, we choose a xhigh-high-xhigh "**reasoning sandwich**" as a baseline.



Spending more reasoning compute on planning and verification

Running only at `xhigh` scored poorly at `53.9%` due to agent timeouts compared to `63.6%` at `high`. There weren't large differences in trial runs across reasoning budget splits so we stuck with our approach which pushed the score to `66.5%`.

The natural approach for models is **Adaptive Reasoning,** seen with Claude and Gemini models where the model decides how much compute to spend on reasoning.

In a multi-model harness, balancing reasoning budgets could play out as using a large model for planning and handing off to a smaller model for implementation.

Practical Takeaways for Building Agent Harnesses

The design space of agents is big. Here are some general principles from our experiments and building deepagents overall.

1. **Context Engineering on Behalf of Agents.** Context assembly is still difficult for agents today, especially in unseen environments. Onboarding models with context like directory structures, available tools, coding best practices, and problem solving strategies helps reduce the error surface for poor search and avoidable errors in planning.
2. **Help agents self-verify their work.** Models are biased towards their first plausible solution. Prompt them aggressively to verify their work by running tests and refining solutions. This is especially important in autonomous coding systems that don't have humans in the loop.

3. **Tracing as a feedback signal.** Traces allow agents to self-evaluate and debug themselves. It's important to debug tooling and reasoning together (ex: models go down wrong paths because they lack a tool or instructions how to do something).

4. **Detect and fix bad patterns in the short term.** Models today aren't perfect. The job of the harness designer is to design around today's shortcomings while planning for smarter models in the future. Blind retries and not verifying work are good examples. These guardrails will almost surely dissolve over time, but to build robust agent applications today, they're useful tools to experiment with.

5. **Tailor Harnesses to Models. T**he Codex and Claude prompting guides show that models require different prompting. A test run with Claude Opus 4.6 scored `59.6%` with an earlier harness version, competitive but worse than Codex because we didn't run the same Improvement Loop with Claude. Many principles generalize like good context preparation and a focus on verification, but running a few rounds of harness iterations for your task helps maximize agent performance across tasks.

There's more open research to do in harness design. Interesting avenues include multi-model systems (Codex, Gemini, and Claude together), memory primitives for continual learning so agents can autonomously improve on tasks, and measuring harness changes across models.

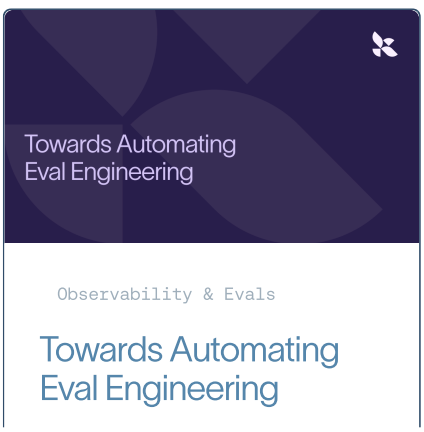
For the outer loop of improving agents, we're looking at methods like RLMs to more efficiently mine traces. We'll be continuing work to improve the harness and openly share our research.

We created a dataset of our Traces to share with the community.

Deep Agents is open source. Python and Javascript.

To more hill climbing and open research.

Related content



How We Benchmark Deep Agents

N. Hollon, H. Chase
July 23, 2026

4 min

Vivek Trivedy
July 22, 2026

5 min

3 Years of Graph Engineering with LangGraph

S. Runkle, H. Chase
July 22, 2026

7 min

Sign up for our newsletter to stay up to date

joh

Subscribe

See what your agent is really doing

LangSmith, our agent engineering platform, helps developers debug every agent decision, eval changes, and deploy in one click.

Try LangSmith

Get a demo

Products

LangSmith Platform
LangSmith Observability
LangSmith Evaluation
LangSmith Deployment
LangSmith Fleet
LangSmith Sandboxes
Deep Agents
LangChain
LangGraph

Resources

Blog
Customer Stories
Guides
Community
Changelog
Docs
Support
LangChain Academy

Company

About
Careers
Partners
Trust Center
Marketing Assets
Events

Sign up for our newsletter to stay up to date

Your email

Subscribe